

For two unit vectors $x, y \in \mathbb{R}^d$ we have that,

$$\frac{1}{c} \cdot \mathbb{E}[\text{d}_H(h(x), h(y))] = \frac{\theta_{x,y}}{\pi}, \quad (3)$$

where $\theta_{x,y} = \arccos(\cos(\theta_{x,y}))$. We do not prove equation 3 in this work; see Theorem §3.1 in [Goemans and Williamson, 1995, Theorem 3.1]. In particular, if x and y are close in angle, the Hamming distance between $h(x)$ and $h(y)$ is low in expectation. Increasing the hash dimension c reduces variance.

The geometric intuition behind this LSH scheme is the following: each row $R_{:,i}$ of R defines a random hyperplane in $\mathbb{R}^d$. The Heaviside function $\text{sgn}(\cdot)$ indicates whether x is positively or negatively oriented with respect to the hyperplane $R_{:,i}$. Thus, the c hyperplanes divide the d dimensional space into multiple partitions, and the resulting c -dimensional hash code is an index into one of the partitions in which x is located. Therefore, vectors with the same or similar hash codes lie in the same or close-by partitions and, therefore, are likely similar in angle. cwcwdsdf

2.1 Related Works

KV Cache Compression Many popular compression strategies adopt an *eviction* approach, which removes embeddings from the KV cache. H₂O [Zhang et al., 2024a] and Scissorhands [Liu et al., 2024a] calculate token importance by their accumulated attention scores and keep the “heavy hitters” in the cache. FastGen [Ge et al., 2023] performs a profiling pass before the generation stage that assigns to each head, according to the head’s attention patterns, a pruning policy which only retains categories of tokens (punctuation, special, etc.) favored by the head. These eviction strategies depend on the computation of attention scores for their policy. An attention-free L_2 dropout method [Devoto et al., 2024], which we compare ourselves to in this work, uses the observation that high-attention tokens tend to have low L_2 key norms to approximately keep important tokens in cache. Other methods seek to merge KV caches across heads, such as grouped query attention (GQA) [Ainslie et al., 2023, Dubey et al., 2024]. KVMerger [Wang et al., 2024] and MiniCache [Liu et al., 2024b], which searches for similarity between tokens in consecutive attention layers and subsequently merges KV cache entries across these layers. While these consolidation approaches prevent memory complexity associated with KV caches from scaling with depth or multi-head attention, the size of any singular cache still tends to scale with sequence length.

LSH Based Attention Similar to our work, Reformer [Kitaev et al., 2020] employs LSH to find similar tokens, but as a way to replace the softmax attention as opposed to token eviction. It creates hash buckets of tokens that form local attention groups and only attends to tokens in the same and neighboring buckets. However, this makes Reformer vulnerable to missing important tokens due to hash collision or boundary issues, and therefore, it must use multiple hash tables to mitigate this issue. In a similar vein, KDEFormer [Zandieh et al., 2023], HyperAttention [Han et al., 2023], and Zandieh et al. [2024a], use LSH to stably approximate and compressing the attention module thus accelerating the computation, but without token eviction. SubGen [Zandieh et al., 2024b] uses LSH to cluster key embeddings and samples representatives from each cluster to reduce the size of the KV Cache and consequently speed up attention, though it must initially view all queries and keys to perform this clustering which could result in VRAM blowup, which our method avoids.

Memory Efficient Transformers Multi-Query Attention [Shazeer, 2019] and Grouped Query Attention [Ainslie et al., 2023] reduce the number of key-value matrices by sharing them across multiple query heads to save KV cache memory usage. However, they require retraining or up-training the LLM. Cache quantization methods [Hooper et al., 2024, Sheng et al., 2023] reduce the KV cache size by compressing the hidden dimension instead of along the sequence dimension but can result in information loss. Linear Transformer [Katharopoulos et al., 2020] reduces memory usage by replacing softmax attention with linear kernels and, therefore, achieves a constant memory requirement.

3 HASHEVICT: A Locality-Sensitive Eviction Strategy

We now formalize our eviction method reflected in Algorithm 1. We assume that the KV cache has a limited and fixed budget and conceptually divide the KV cache management during LLM inference into two stages: the initial Prompt Encoding Stage and then a Decoding Stage (i.e., generation).

Let C be a constant and fixed cache budget, $\mathcal{K}$ be the key cache, and $\mathcal{V}$ be the V cache in a K-V attention head. We define our eviction policy as a function

$$\mathcal{K}_t, \mathcal{V}_t, \mathcal{H}_t \leftarrow P(q, \mathcal{K}_{t-1}, \mathcal{V}_{t-1}, \mathcal{H}_{t-1}) \quad (4)$$

where $\mathcal{H}_t \in \{0, 1\}^{b \times C}$ is a hash table that contains hash codes of keys in $\mathcal{K}$. We then define a function F_{score} to assign a score for each key inside the K cache. F_{score} outputs an array which contains the negative of hamming distances d_H

between the hash code of a query vector q and columns of $\mathcal{H}$, which are hash codes of all non-evicted keys.

$$F_{score}(q, \mathcal{K}) = -d_H(h(q), \mathcal{H}) \quad (5)$$

The eviction index e_t at any step t is selected as the index with the lowest score:

$$e_t \leftarrow \arg \min F_{score}(q_{t-1}, \mathcal{H}_{t-1}) \quad (6)$$

which points to the key that is most distant from the query vector at time step t . Entries at index e_t from the $\mathcal{K}$ and $\mathcal{V}$ are evicted and $\mathcal{H}$ is updated (step 3-6 of Algorithm 1).

Algorithm 1 HASHEVICT (timestep t)

Require: query q , key k , value v , key cache $\mathcal{K}$, value cache $\mathcal{V}$, hash table $\mathcal{H}$

- | | |
|---|--|
| 1: $e_t \leftarrow \arg \min F_{score}(q_t, \mathcal{H}_{t-1})$ | ▷ Determine eviction index e_t |
| 2: del $\mathcal{K}_{t-1}^{e_t}, \mathcal{V}_{t-1}^{e_t}, \mathcal{H}_{t-1}^{e_t}$ | ▷ Remove entries at index e_t from KV cache and hash table |
| 3: $\mathcal{K}_t \leftarrow \mathcal{K}_{t-1} \cup k_t$ | ▷ Update key cache |
| 4: $\mathcal{V}_t \leftarrow \mathcal{V}_{t-1} \cup v_t$ | ▷ Update value cache |
| 5: $\mathcal{H}_t \leftarrow \mathcal{H}_{t-1} \cup h(k_t)$ | ▷ Add hash of k_t to the hash table |
| 6: $A \leftarrow \text{Attention}(q, \mathcal{K}_t, \mathcal{V}_t)$ | ▷ Calculate attention |
-

Prompt Encoding Stage During the prompt encoding stage, the model processes the prompt, $x_{prompt} = [x_1, \dots, x_N] \in \mathbb{R}^{N \times d}$. The KV cache and the hash table are first filled to full by the C tokens. $\mathcal{K}_0 = \{k_1, \dots, k_C\}, \mathcal{V}_0 = \{v_1, \dots, v_C\}, \mathcal{H}_0 = h(\mathcal{K}_0) = \bigcup_{i \in [1, C]} h(k_i)$. We then set $t \leftarrow C + 1$, and begin Algorithm 1.

Decoding Stage Let $x_{decoding} = [z_1, \dots, z_T] \in \mathbb{R}^{T \times d}$ be the generated tokens during auto-regressive decoding. In the decoding stage, we continue Algorithm 1 by setting $t \leftarrow -N + 1$. The generation completes at time step $N + T$.

Complexity Our strategy assumes a fixed memory budget, and therefore, uses constant memory. The computation overhead per time step is also constant, because F_{score} is calculated for a constant C number of key vectors in the cache. The extra memory overhead that HASHEVICT introduces to each attention head is the hash table $\mathcal{H}$, which only uses $C * b$ bits of space and is independent of the sequence length. The hash table is stored on GPU memory and does not introduce any latency bottlenecks associated with CPU-to-GPU streaming [Strati et al., 2024].

4 Experiments

Tasks We evaluated HASHEVICT across various tasks to demonstrate its effectiveness in reducing the memory cost of the KV cache while preserving the language quality of the generated text. Our experiments are split into four main categories: free response question answering, multiple choice, long-context retrieval and long-context summarization. Our long context retrieval tasks include the multi-key needle-in-a-haystack task and the common words task from [Hsieh et al., 2024]. Question answering tasks include GSM8K [Cobbe et al., 2021] and MedQA [Jin et al., 2021]. Summarization tasks include GovReport and MultiNews from Bai et al. [2023].

Metrics The question-answering tasks were evaluated using BERTScore (which includes precision, recall, and F1 scores), ROUGE (ROUGE-1, ROUGE-2 and ROUGE-L and ROUGE-Lsum), and GPT4-Judge. GPT-4 was prompted to look at both the model prediction and the ground truth answer, then provide a score from 1 - 5 on the coherence, faithfulness, and helpfulness of the answer in addition to similarity between the prediction and ground truth (we named this metric GPT4-Rouge). In this section, we report the average of these four scores. For details on individual scores, please see Appendix A. For the system prompts given to GPT-4, refer to Appendix F.2. For multiple-choice tasks, we use accuracy as our metric. The metric used to evaluate long context retrieval tasks is the string matching score from Hsieh et al. [2024], whose definition is in Appendix F.1. For summarization tasks, we use Rouge as the metric as per direction from Bai et al. [2023].

Configuration and Setup We conducted most experiments using Meta’s Llama3 8B-Instruct model [Dubey et al., 2024] with the exception of long text summarization tasks which were tested using the Llama3.1 8B-Instruct model. Our method is agnostic to grouped-query attention, so we used the default group size of 4. The maximum sequence length was set to the sum of the maximum prompt length and the maximum number of allowed generated tokens needed for each task. We conducted experiments using cache budgets of 10%, 30%, 50%, 70%, and 90% of the full KV cache.

Based on insights from [Xiao et al., 2023, Child et al., 2019, Beltagy et al., 2020], we also keep the most recent 10 tokens and the first 4 tokens of the prompt always in the KV cache. The summarization tasks were performed on Nvidia H100 80GB graphics cards due to their long contexts. All other experiments were conducted on the Google Cloud Platform G2 instances with Nvidia L4 24GB graphics cards.

Baseline Methods We chose the L_2 norm-based eviction method [Devoto et al., 2024] as our main baseline for comparison because it is also an attention-free KV cache eviction method. We also included two attention-accumulation-based methods: H₂O Zhang et al. [2024a] and Scissorhands Liu et al. [2024a], as well as a hybrid method: Fastgen Ge et al. [2023].

4.1 Free Response Question Answering

We tested our strategy against tasks that require generating accurate answers using multi-step reasoning. Specifically, we used the GSM8K and MedQA datasets to assess language quality for each strategy, given a constrained KV cache budget. Both tasks are used to test the potential side effects of compression on the LLM’s reasoning ability.

GSM8K GSM8K consists of grade-school-level math problems that typically require multiple reasoning steps. As shown in Figure 2, HASHEVICT strategy consistently outperforms the L_2 norm-based method across various cache sizes. Notably, even when the KV cache budget is set to 50% of the full capacity, the HASHEVICT eviction strategy maintains a high answer quality, with minimal degradation in BERTScore F1, ROUGE-L, and GPT4-Judge scores. Additionally, HASHEVICT performs on par with H₂O and Scissorhands without accumulating attention scores.

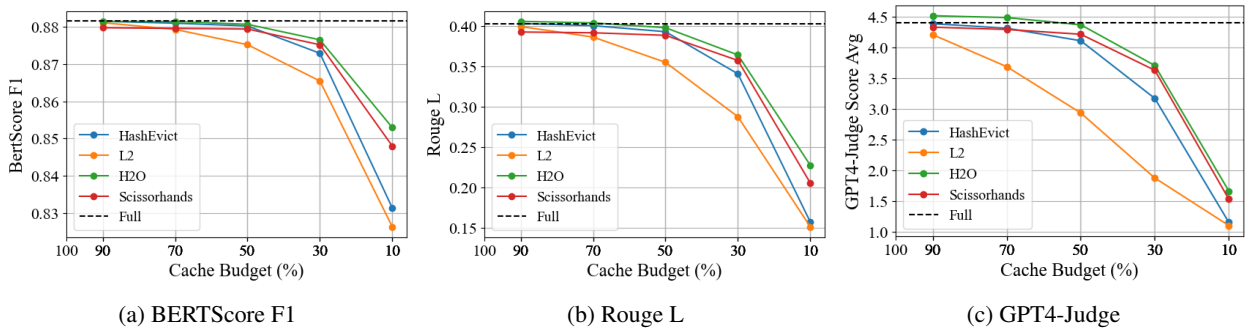


Figure 2: **GSM8K Question Answering Performance.** We measure BERTScore F1, Rouge-L, and GPT4-Judge for different cache budgets on a grade school math task. HASHEVICT outperforms L_2 for all three metrics for every budget, with sharp differences for the 50% and 30% compression. HASHEVICT performs similarly to H₂O and Scissorhands except at 10% cache budget.

MedQA MedQA is a free response multiple choice question answering dataset collected from professional medical board exams. We randomly sampled 100 questions from this dataset. Each question has 5 choices and only one correct answer, along with ground truth explanations and reasoning steps. Figure 3 illustrates that HASHEVICT performs better than all baseline methods for all cache budgets tested. For both datasets, HASHEVICT produced more coherent and helpful answers across all cache budgets than the baselines per Table 9.

For detailed experiment results of both question answering tasks, and for comparison with Fastgen at various attention recovery ratios, please refer to Appendix A.

4.2 Multiple Choice Question Answering

We evaluated our method on multiple-choice versions of GSM8K and MedQA. Multiple choice is a more difficult test of a model’s reasoning capability under the constraint of cache compression, as it takes away the ability to use intermediate results in the generated text. The model has to keep useful tokens during prompt compression in order to pick the correct answer choice.

GSM8K Multiple Choice For the GSM8K multiple choice experiments, HASHEVICT significantly outperforms L_2 for cache budgets of 30% and 50%. As shown in Figure 4a, the L_2 method’s accuracy drops significantly at smaller cache sizes, while the performance of HASHEVICT does not significantly drop until the cache budget is set at 10%.